

Quadruped Locomotion with Central Pattern Generators and Deep Reinforcement Learning

Frederik Kaiser, Tomasz Niedziałkowski, Grzegorz Pawlak

Group 7

EPFL, Legged Robots – Miniproject 2

I. INTRODUCTION

A. Background and Motivation

Legged robots have the potential to operate in environments that are inaccessible to traditional wheeled or tracked platforms, such as rough natural terrain, cluttered industrial sites, or disaster areas. Among them, quadrupeds offer a compelling compromise between stability, manoeuvrability, and mechanical complexity. However, achieving robust and efficient locomotion on such systems remains challenging due to their high-dimensional, nonlinear dynamics and the need to coordinate multiple contacts with the environment in a dynamically consistent way.

A broad range of control strategies has been proposed to address this problem. On one side, bio-inspired approaches rely on central pattern generators (CPGs) to produce rhythmic joint or foot trajectories that encode locomotion gaits in a compact and interpretable form. Properly tuned, CPGs can generate smooth and repeatable motions, and they provide intuitive handles for adjusting gait properties such as step length, duty factor, and walking speed. On the other side, model-free deep reinforcement learning (RL) has emerged as a powerful framework for discovering locomotion behaviours directly from trial-and-error interaction in simulation. By optimising a task-specific reward under an appropriate Markov decision process (MDP) formulation, RL can exploit complex feedback and contact interactions that are difficult to capture with hand-designed controllers.

B. Project Overview

This mini-project investigates quadruped locomotion in simulation by combining these two viewpoints. In Part 1, we design and implement CPG-based controllers for a simulated quadruped robot, using coupled Hopf oscillators to generate Cartesian foot trajectories for a trot gait. These trajectories are tracked through joint-space and Cartesian-space PD control, allowing us to study how CPG parameters and control modes influence the resulting body motion, duty cycle, and energy usage. In Part 2, we formulate quadruped locomotion as an RL problem and train policies that track desired forward velocities and solve a task-specific terrain challenge, such as negotiating gaps, slopes, or stairs.

C. Contributions and Report Structure

The goal of this report is not only to obtain working locomotion behaviours, but also to analyse and compare the design choices behind them. We therefore present the modelling and control formulations used for both the CPG-based and RL-based controllers, and we evaluate their performance using quantitative metrics such as average velocity, cost of transport, and gait properties, as well as qualitative observations of robustness on challenging terrain. The remainder of the report is organised as follows. Section II describes the quadruped model and control interfaces used in simulation, and details the methodology for both the CPG-based controllers and the deep RL setup. Section III presents the experimental results for both parts of the project. Section IV discusses the observed trade-offs between CPG and RL approaches, and Section V concludes with a summary and perspectives for future work.

II. METHODOLOGY

A. Quadruped Modeling and Control Modes

1) *Coordinate frames and indexing*: We simulate a quadruped robot in PyBullet with four legs, each actuated by three joints (Hip, Thigh, Calf), for a total of 12 motors. Consequently, when querying joint states or sending motor torques, we use arrays of length 12. The legs are indexed as Front Right (FR, 0), Front Left (FL, 1), Rear Right (RR, 2), and Rear Left (RL, 3), and the joint order within each leg is always Hip–Thigh–Calf. In many parts of the controller, we work in the *leg frame* (a local reference frame attached to each hip), because inverse kinematics and Jacobian computations are naturally expressed in this frame. As a reference configuration, a neutral standing pose places the foot directly under the hip at a nominal body height of 0.25 m (e.g., for the FR leg one example is $(x, y, z) = (0, -\text{hip length}, -0.25)$ m in the leg frame).

2) *Kinematics and Jacobian relations*: For each leg i , the forward kinematics map the joint angles $q \in \mathbb{R}^3$ to the Cartesian foot position $p \in \mathbb{R}^3$:

$$p = f(q). \quad (1)$$

Differentiating yields the foot linear velocity

$$v = \dot{p} = J(q)\dot{q}, \quad (2)$$

where $J(q) \in \mathbb{R}^{3 \times 3}$ is the single-leg Jacobian. In addition, the Jacobian transpose provides a standard mapping from desired Cartesian end-effector forces F to joint torques:

$$\tau = J(q)^T F. \quad (3)$$

These relationships enable switching between joint-space tracking and task-space (Cartesian) tracking in a consistent way.

3) *Low-level tracking controllers:* We use proportional-derivative (PD) control in joint space and, optionally, an additional PD term in Cartesian space.

a) *Joint-space PD control:* Given desired joint trajectories $(q_d, \dot{q}_d)$ and measured states $(q, \dot{q})$, the joint-space torque command is

$$\tau_{\text{joint}} = K_{p,\text{joint}}(q_d - q) + K_{d,\text{joint}}(\dot{q}_d - \dot{q}), \quad (4)$$

where $K_{p,\text{joint}}$ and $K_{d,\text{joint}}$ are (typically diagonal) gain matrices.

b) *Cartesian-space PD control:* To improve end-effector tracking, we compute desired foot trajectories (p_d, v_d) in the leg frame (e.g., from a Cartesian target mapped through inverse kinematics). With measured foot states (p, v) , the Cartesian PD term is

$$\tau_{\text{cart}} = J(q)^T (K_{p,\text{cart}}(p_d - p) + K_{d,\text{cart}}(v_d - v)), \quad (5)$$

where $K_{p,\text{cart}}$ and $K_{d,\text{cart}}$ are Cartesian gain matrices.

c) *Combined command:* When both terms are enabled, we sum them to obtain the final motor torques:

$$\tau_{\text{final}} = \tau_{\text{joint}} + \tau_{\text{cart}}. \quad (6)$$

This unified formulation allows us to directly compare tracking performance with joint PD alone versus joint+Cartesian PD in the experimental evaluation.

B. Part 1 – CPG-Based Locomotion

We implement open-loop locomotion using a Central Pattern Generator (CPG) network that outputs desired foot trajectories, which are tracked in torque control using joint-space PD with an optional Cartesian PD term. All experiments are executed in PyBullet with a simulation time step of 1 ms (control update every step, `action_repeat = 1`) and motor control mode set to `TORQUE`.

1) *CPG Formulation:* We assign one Hopf oscillator to each leg $i \in \{\text{FR, FL, RR, RL}\}$. Each oscillator state is represented in polar form by an amplitude r_i and a phase θ_i , and the coupled network converges to a stable limit cycle. In our implementation, the oscillator amplitude converges towards $\sqrt{\mu}$ with convergence rate α . For the trot gait experiments reported here, we use $\mu = 1$ and $\alpha = 50$.

We employ a swing/stance phase split based on θ_i :

$$\text{swing} : 0 \leq \theta_i \leq \pi, \quad \text{stance} : \pi < \theta_i < 2\pi, \quad (7)$$

and the oscillator evolves with different angular frequencies in swing and stance, denoted ω_{swing} and ω_{stance} , respectively. The CPG internally outputs two signals per leg: (x_i, z_i) , which directly parameterise the desired foot motion in the sagittal plane.

2) *Gait Design and Coupling:* The gait is set by the Hopf network configuration (here: `TROT`) with oscillator coupling enabled (`couple=True`). Coupling enforces consistent inter-leg phase relationships characteristic of the chosen gait. The resulting coordination pattern is fully determined by the gait configuration and the oscillator parameters.

3) *Mapping to Foot Trajectories:* At every control step, the Hopf network returns desired sagittal foot coordinates (x_i, z_i) for each leg i . We define the desired foot position in the leg frame as

$$p_d^{(i)} = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}, \quad (8)$$

where the lateral coordinate is kept constant to maintain nominal stance width. Specifically, we fix

$$y_i = s_i \ell_{\text{hip}}, \quad \ell_{\text{hip}} = 0.0838 \text{ m}, \quad s = [-1, +1, -1, +1], \quad (9)$$

so that legs on the right side use negative hip sign and legs on the left side use positive hip sign.

Given $p_d^{(i)}$, we compute desired joint angles via inverse kinematics:

$$q_d^{(i)} = \text{IK}(i, p_d^{(i)}). \quad (10)$$

In our controller implementation, desired joint velocities are set to zero (i.e., $\dot{q}_d^{(i)} = 0$), and damping is applied using measured joint velocities.

4) *Joint and Cartesian PD Tracking:* Let $q^{(i)}$ and $\dot{q}^{(i)}$ denote measured joint angles and velocities for leg i . We compute torque commands as the sum of a joint-space PD term and (optionally) a Cartesian PD term.

a) *Joint-space PD:* For each leg i , the joint PD torque is

$$\tau_{\text{joint}}^{(i)} = K_{p,\text{joint}}(q_d^{(i)} - q^{(i)}) + K_{d,\text{joint}}(0 - \dot{q}^{(i)}), \quad (11)$$

with gains (identical for all legs)

$$K_{p,\text{joint}} = \text{diag}(100, 100, 100), \quad K_{d,\text{joint}} = \text{diag}(2, 2, 2). \quad (12)$$

b) *Cartesian PD:* When `ADD_CARTESIAN_PD=True`, we augment the torque with a Cartesian correction computed in the leg frame. We first obtain the current Jacobian and foot position:

$$J^{(i)}, p^{(i)} = \text{JacobianAndPosition}(i), \quad (13)$$

and compute the foot linear velocity using the Jacobian:

$$v^{(i)} = J^{(i)} \dot{q}^{(i)}. \quad (14)$$

The desired Cartesian position is taken as the forward-kinematics position corresponding to the IK solution (to ensure consistency between IK and task space):

$$p_{d,\text{fk}}^{(i)} = \text{Position}(i, q_d^{(i)}), \quad (15)$$

and the desired Cartesian velocity is set to zero ($v_d^{(i)} = 0$). The Cartesian PD torque is then

$$\tau_{\text{cart}}^{(i)} = \left(J^{(i)} \right)^T \left(K_{p,\text{cart}}(p_{d,\text{fk}}^{(i)} - p^{(i)}) + K_{d,\text{cart}}(0 - v^{(i)}) \right), \quad (16)$$

with gains

$$K_{p,\text{cart}} = \text{diag}(500, 400, 400), \quad K_{d,\text{cart}} = \text{diag}(30, 50, 40) \quad (17)$$

c) *Final command*: The final leg torque is

$$\tau_{\text{final}}^{(i)} = \tau_{\text{joint}}^{(i)} + \tau_{\text{cart}}^{(i)}, \quad (18)$$

and the 12D action vector concatenates torques for all legs in Hip–Thigh–Calf order.

5) *Hyperparameters and experimental protocol*: For the trot gait results, we run each simulation for 5 s and log the CPG states $(r, \theta, \dot{r}, \dot{\theta})$, desired/actual foot positions, and desired/actual joint angles. To avoid initial transient effects (startup drift/slip), quantitative tracking plots and RMS errors are computed over a steady-state window after the robot has already reached continuous walking.

C. Part 2 – Deep Reinforcement Learning

This section draws inspiration from the work of Bellegarda et al. [1]. The proposed controller utilizes Central Pattern Generators (CPGs) as a baseline, reusing the CPG implementation detailed in Part 1. By integrating CPGs into the RL framework, we aim to guide the learning process with rhythmic priors while allowing the policy to adapt the gait parameters.

1) *Observation Space*: We selected the full observation space configuration described in [1]. Based on the reference paper’s analysis of observation complexity, we utilize a comprehensive set of proprioceptive measurements to maximize performance. These include the body state (orientation, linear, and angular velocities), joint state (positions and velocities), and Boolean foot contact indicators.

Additionally, the observation vector includes the last action chosen by the policy network and the CPG states $\{r, \dot{r}, \theta, \dot{\theta}\}$. To enable velocity tracking capabilities, velocity commands were added to the feature set, allowing the robot to walk at variable speeds. The complete specification of the observation space is presented in Table II in Appendix A.

2) *Action Space*: The action space was kept consistent with the CPG-based controller. It consists of a vector of 8 continuous values in the range $[-1, 1]$. These values modulate the intrinsic CPG parameters $\{\mu_{1..4}, \omega_{1..4}\}$, which are subsequently mapped to joint torque commands via a joint-space PD controller. Virtual Model Control (VMC) was initially tested within the pipeline; however, as it did not yield significant performance improvements, further development proceeded without this module.

3) *Reward Function*: The reward function is based on the formulation presented in [1]. We investigated different levels of complexity by testing the function components from the bottom up. The complete reward function consists of terms for forward velocity tracking, lateral velocity minimization, yaw rate tracking, vertical stability penalty, roll/pitch rate penalties, and an energy penalty to minimize motor torques.

The specific weights and formulas used are detailed in Table III in Appendix A. The main components responsible for tracking the task objective are mapped to the range $[0, 1]$ using

a Gaussian kernel and weighted accordingly. This clipping ensures smooth gradients and improves learning stability. Critically, we penalize rates of change (e.g., angular velocity) rather than absolute values (e.g., orientation angle) to allow the robot to tilt on slopes or stairs without incurring large penalties.

4) *RL Algorithms and Hyperparameters*: Proximal Policy Optimization (PPO) was selected as the RL algorithm due to its balance of sample efficiency and stability, implemented via the Stable Baselines 3 framework. The hyperparameters used for training are presented in Table IV in Appendix A.

Policies were trained for either 3 or 5 million steps using 40 parallel environments. Training for 3 million steps typically required 1 hour, while 5 million steps required approximately 1.5 hours on an Intel Xeon Gold 6226R CPU. We also investigated an adaptive learning rate approach similar to [2], which adjusts the step size based on the KL-divergence to prevent policy collapse.

5) *Environment Modifications and Training Setup*: To improve policy robustness and facilitate Sim-to-Real transfer, we employed domain randomization. Ground friction and observation noise were randomized to mimic real-world sensor imperfections. Additionally, CPG parameters and target velocities were randomized during training:

- Target velocity was sampled uniformly from $[0.3, 0.8]$ m/s every 5 seconds.
- Desired body height and step height were sampled once per episode.

For the task-specific policy, we implemented curriculum learning to gradually increase the difficulty of terrain features. For slopes, the pitch angle is increased; for stairs, strategies included increasing step height or the number of steps over time.

III. RESULTS

A. Part 1 – CPG Results

1) *Gait Performance and Hyperparameter Tuning*: To achieve stable locomotion across varying velocities, we performed an extensive hyperparameter search focused on the Trot gait. The primary parameters governing forward velocity (v) in our CPG framework are the stride frequency ($f = \frac{\omega}{2\pi}$) and the step length (d_{step}). Theoretically, the forward velocity can be approximated as $v \approx 2 \cdot f \cdot d_{\text{step}}$ for a trot gait, though slippage and motor dynamics introduce non-linearities.

We identified two distinct configurations representing the stable performance limits of our controller: a slow-speed precision mode and a high-speed dynamic mode. The specific hyperparameters and resulting gait metrics are summarized in Table I.

For the **slowest stable gait** (0.29 m/s), we found it necessary to reduce the step length significantly to 0.05 m. While intuitively one might simply reduce the frequency, we observed that frequencies below 0.3 Hz caused lateral instability, as the robot spent too long in a generic unstable equilibrium during the swing phase of the diagonal leg pairs.

TABLE I: Hyperparameter Configurations for Trot Extremes

Parameter	Slow Configuration	Fast Configuration
μ (Amplitude)	1.0	1.0
ω_{swing}	$0.6\pi \approx 1.88 \text{ rad/s}$	$4.0\pi \approx 12.57 \text{ rad/s}$
ω_{stance}	$0.6\pi \approx 1.88 \text{ rad/s}$	$4.0\pi \approx 12.57 \text{ rad/s}$
Step Length (d_{step})	0.05 m	0.14 m
α	50	50
Resulting Metrics		
Avg. Velocity	0.29 m/s	1.28 m/s
Duty Cycle (D)	0.50	0.50
Cycle Period (T)	$\approx 3.33 \text{ s}$	$\approx 0.50 \text{ s}$
Cost of Transport	1.18	3.13

For the **fastest stable gait** (1.28 m/s), we increased the frequency to 2.0 Hz and the step length to 0.14 m. Beyond these values, we observed kinematic singularities where the desired foot position exited the workspace of the leg, and torque saturation in the hip motors which led to tracking errors and eventual falls.

The resulting **Duty Cycle** ($D = \frac{T_{\text{stance}}}{T_{\text{swing}} + T_{\text{stance}}}$) remained at 0.5 for both configurations, as we maintained symmetric swing and stance frequencies ($\omega_{\text{sw}} = \omega_{\text{st}}$).

Regarding energy efficiency, we calculated the mechanical Cost of Transport (CoT) as $\text{CoT} = \frac{E}{mgd}$. As shown in Table I, the CoT increased significantly at higher velocities (1.18 vs 3.13). This indicates that for our controller, the dynamic costs—specifically the high joint torques required to accelerate and decelerate the legs rapidly (2 Hz) against damping and inertia—grow faster than the gain in forward distance. Conversely, the slow gait proved more mechanically efficient, likely because the joint velocities were minimal, reducing the power contribution from the $\tau \cdot \dot{q}$ term despite the longer duration required to traverse the same distance.

2) *Proposed Extensions and Feedback Mechanisms*: While the current CPG implementation relies on open-loop oscillator coupling, the biological systems inspiring this approach heavily utilize sensory feedback. We propose three major extensions to enhance robustness and controllability:

3) *Virtual Model Control (VMC) and Postural Feedback*: The current controller does not actively regulate the trunk orientation, which leads to drift when traversing uneven terrain. As proposed by Ajallooeian et al. [1], [3], a Virtual Model Control layer could be superimposed on the CPG output. This involves computing virtual forces and torques required to keep the torso level (IMU feedback) and mapping them to joint torques via the Jacobian transpose (J^T). This would effectively act as a "reflex" mechanism, adjusting leg stiffness during the stance phase to reject pitch and roll disturbances.

4) *Phase Modulation via Force Feedback ("Tegotae")*: To improve robustness against stumbling, the phase dynamics of the oscillators could be modulated by ground reaction forces, a concept known as "Tegotae" [?]. Instead of a fixed time-based phase evolution ($\dot{\theta} = \omega$), we can introduce a feedback

term:

$$\dot{\theta}_i = \omega_i + \text{coupling} + f(\text{GRF}_i) \quad (19)$$

where $f(\text{GRF}_i)$ prevents the phase from advancing to "swing" if the leg is still heavily loaded (bearing weight). This ensures that a leg does not lift prematurely if the robot is tilting towards it, significantly improving stability on slopes.

5) *Descending Control Signals*: Currently, gait parameters are hard-coded. A hierarchical control architecture could be implemented where a high-level planner (or human operator) sends descending command signals $\mathbf{u} = [v_x, v_y, \omega_z]$ (forward, lateral, and turning velocities). These commands would map directly to CPG parameters:

- $v_x \rightarrow$ modulates step length d_{step} and frequency ω .
- $v_y \rightarrow$ introduces a phase shift or abduction bias to enable strafing.
- $\omega_z \rightarrow$ creates asymmetric stride lengths between left and right legs to induce turning.

This maps closely to the CPG-RL approach explored in Part 2, where the RL policy effectively acts as the descending controller modulating these parameters.

B. Part 2 – RL Velocity Controller

As mentioned in Section II, multiple policy configurations were tested for the velocity controller. This section analyzes the impact of different components on performance.

1) *Main Policy Performance*: The best overall performance was achieved by the policy utilizing the extended observation space (including desired velocity), a simplified custom reward function (velocity tracking dominant), and an adaptive learning rate. This policy was trained with a desired X-velocity of 0.8 m/s and Y-velocity of 0 m/s. CPG parameters were randomized with a desired body height in $[0.2, 0.3]$ m and maximum foot height in $[0.03, 0.2]$ m. The policy was trained for 3 million steps.

Contrary to expectations, the simplest reward function yielded the superior results. While policies trained with orientation constraints resulted in a slightly more stable posture, the robot frequently slowed down to maintain balance. The basic reward function allowed for high agility and consistent velocity tracking. The inherent stability provided by the CPG baseline likely mitigates the need for strict orientation penalties. The Cost of Transport (CoT) for this policy was calculated at 1.9. Figure 1 illustrates the training progress; although trained for 3 million steps, the policy converged after approximately 1.5 million steps.

2) *Trajectories and Robustness*: The CPG states recorded during inference are presented in Figure 2, and the resulting leg trajectories are shown in Figure 3. The policy demonstrates the ability to adapt to changes in CPG parameters, maintaining stable locomotion even when parameters vary dynamically. This robustness is critical; initially, setting the desired body height lower than 0.2 m caused the robot's chassis to contact the ground, leading to frequent episode terminations. Adjusting the sampling range to $[0.2, 0.3]$ m resolved this.

3) **Ablation Studies: Observation Space:** The initial implementation of the observation space lacked the commanded X-velocity. Consequently, the policy was unaware of changes in the task objective. As shown in Figure 4, the policy without velocity commands could only learn to keep the robot standing but failed to track velocity targets effectively.

PD Gains: We observed that higher PD gains ($k_p = 100$) resulted in stiff motion but better stability on flat ground. Reducing gains by 50% caused the policy to fail completely when trained from scratch. Curriculum learning (sequentially reducing gains by 15%, 30%, 50%) produced a functional but less stable policy. Thus, the stiffer setup was preferred. Figure 5 compares the mean rewards and episode lengths.

Reward Function Components: We tested various reward configurations by removing components sequentially (Figure 6). All variations converged to similar final rewards, but the simplest function was selected for its flexibility, particularly regarding orientation changes required for the task-specific controller.

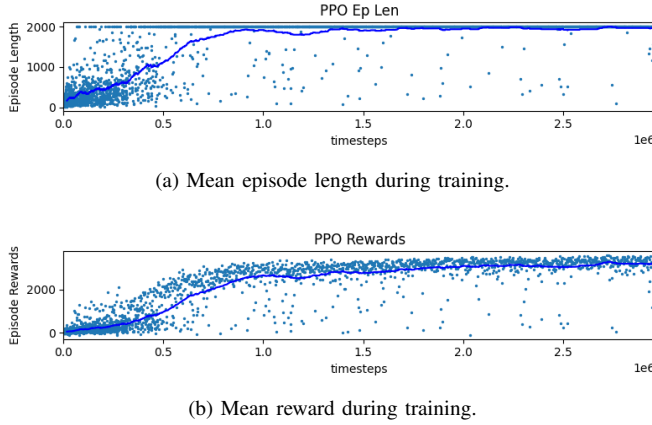


Fig. 1: Training progress for the velocity controller. (a) Episode length increases as the agent learns to survive. (b) Mean reward converges during training.

C. Part 2 – RL Task-Specific Controller

For the task-specific controller, we selected slope traversal as the primary objective. The complexity of this obstacle serves as a suitable benchmark before attempting stairs, which were not fully trained due to time constraints. We selected the configuration with the simplest custom reward function (from the velocity controller) due to its high performance and flexibility regarding body orientation, which is crucial for adapting to non-flat terrain.

All other MDP properties, network hyperparameters, and hardware configurations remained consistent with Part 1. We compared the policy trained with our simple custom reward against a policy trained with the reward function from the CPG-RL paper [1]. Additionally, we investigated the reusability of the velocity controller policy via fine-tuning and the effectiveness of curriculum learning.

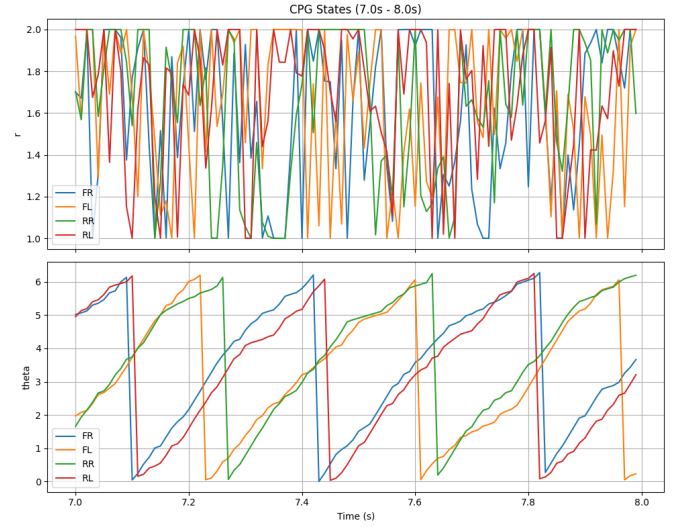


Fig. 2: CPG states for 1 second of a deployed policy (main velocity controller) locomoting at 0.8 m/s.

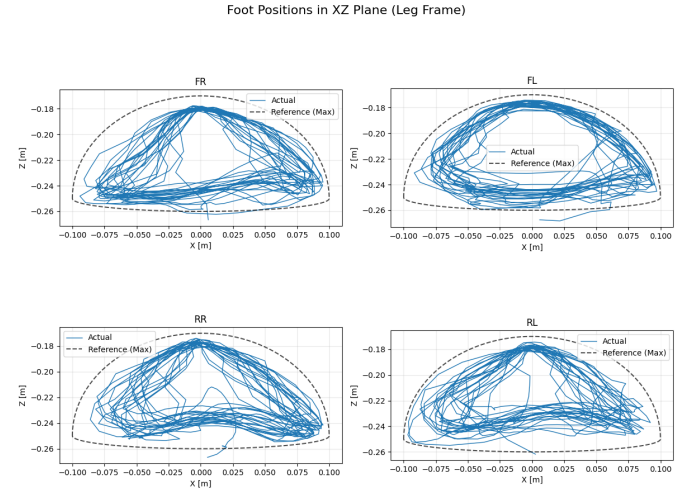
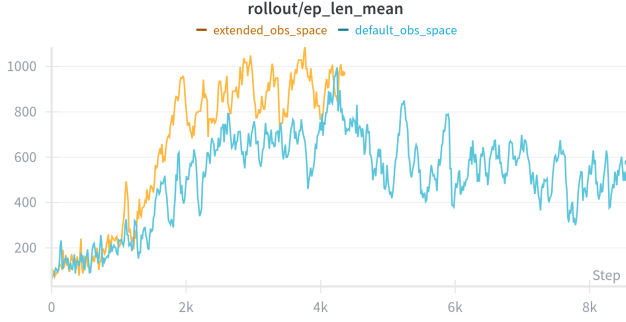


Fig. 3: Leg-frame foot XZ trajectories for a deployed policy locomoting at 0.8 m/s.

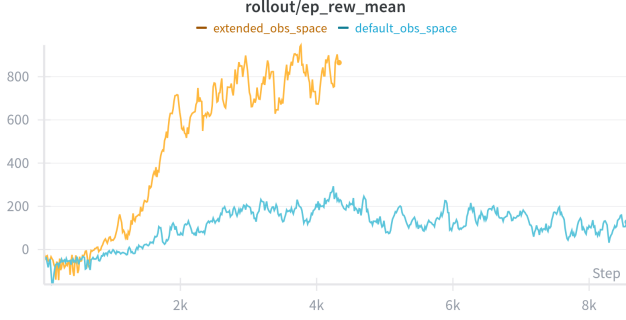
1) **Training Setup:** The policies were trained for 3×10^6 steps. The curriculum learning rule included three difficulty levels: for every 1 million steps, the pitch was increased by 0.1 rad, up to a maximum pitch of 0.2 rad.

2) **Performance Analysis:** The best results were achieved by policies trained from scratch using domain randomization (curriculum), with both the CPG-RL reward and our simple custom reward. As shown in the training curves in Figure 9, both policies exhibited similar performance characteristics.

Qualitatively, both policies are able to traverse the slope in a stable manner. They both exhibit minor difficulties in the initial stage of climbing, where the robot must adjust its posture to the changed pitch of the ground. However, they handle terrain transitions (flat-to-slope and slope-to-downhill) very



(a) Mean episode length.



(b) Mean reward.

Fig. 4: Comparison of observation space configurations. The limited observation space prevented the agent from distinguishing velocity targets.

consistently. Crucially, the descent phase is handled reliably; as seen in Figure 12, the robot accelerates only marginally during descent. There is no significant slippage, and the stance feet maintain their position. While occasional missteps at the slope edge caused orientation deviations in some experiments, the policy successfully stabilized the body after a few steps and continued trotting.

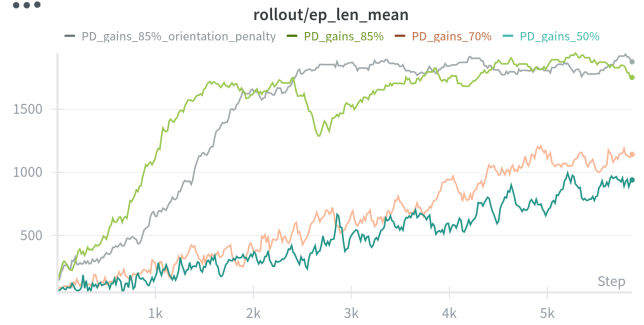
3) *Curriculum and Fine-tuning*: Interestingly, training the policy without curriculum learning did not significantly hinder performance. The resulting policy behaved only marginally worse than those trained with the curriculum, suggesting that the maximum pitch of 0.2 rad might have been small enough for the agent to learn directly.

Regarding fine-tuning, the policy initialized with the pre-trained velocity controller model did not yield positive outcomes. Although the initial training phase was similar to the scratch models, the policy performance plateaued early and failed to improve further (see Figure 13).

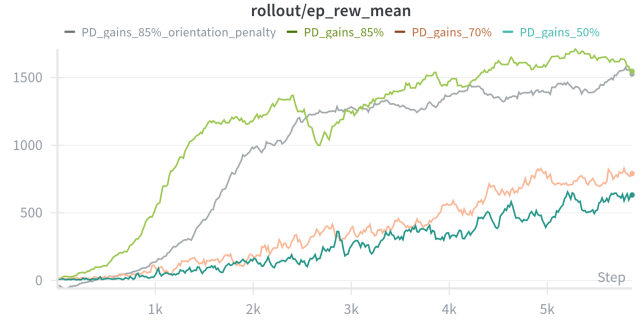
IV. DISCUSSION

A. Future Work

Further work would include training on a wider variety of obstacles. However, to achieve this, successful fine-tuning strategies must be developed, as training from scratch for complex tasks is computationally expensive. According to the

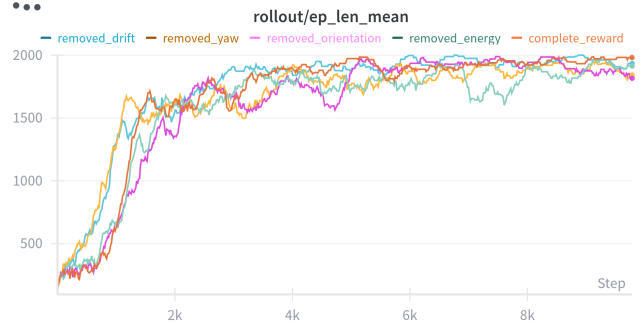


(a) Mean episode length.

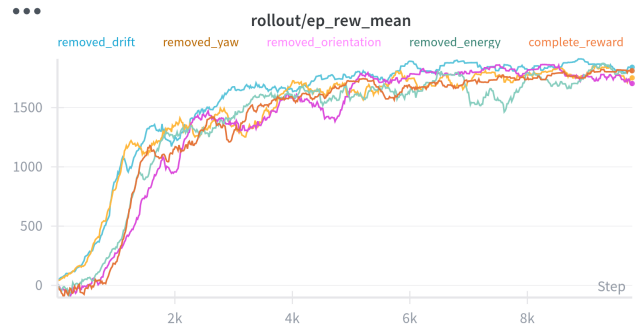


(b) Mean reward.

Fig. 5: Comparison of PD gains performance.



(a) Mean episode length.



(b) Mean reward.

Fig. 6: Comparison of custom reward function components.

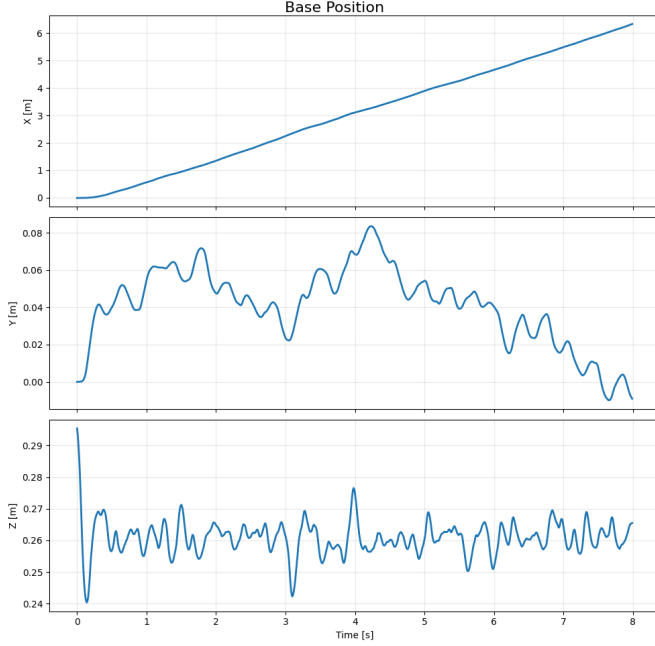


Fig. 7: Base position for a deployed policy locomoting at 0.8 m/s.

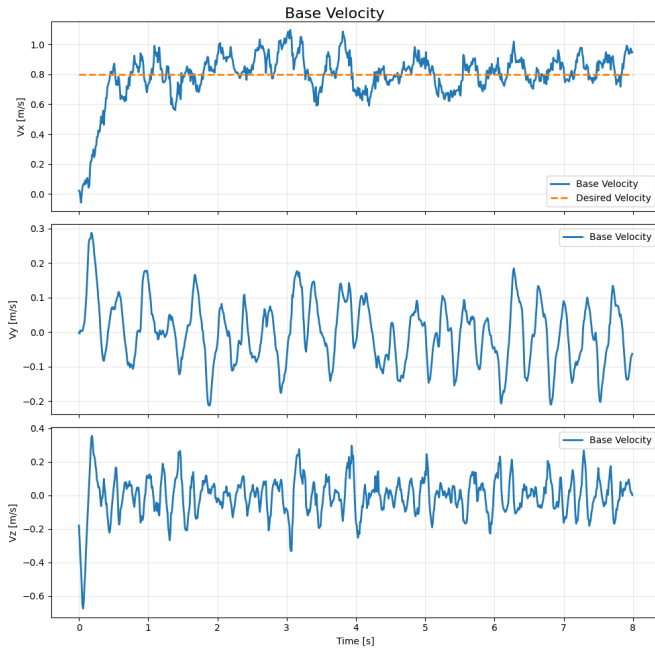
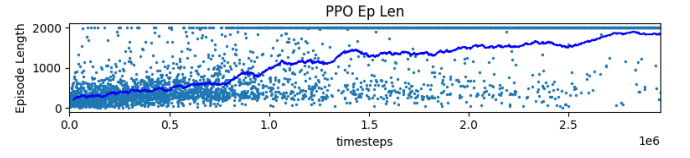
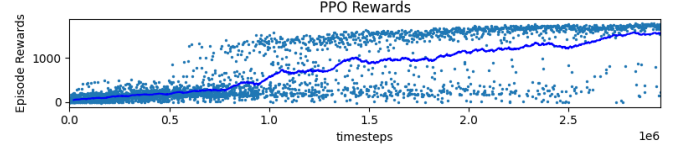


Fig. 8: Base velocity for a deployed policy locomoting at 0.8 m/s.



(a) Mean episode length during training.



(b) Mean reward during training.

Fig. 9: Training progress for the task-specific controller. (a) Episode length increases as the agent learns to survive. (b) Mean reward converges. Compared to the velocity controller, more episodes end prematurely, but the mean length approaches the maximum.

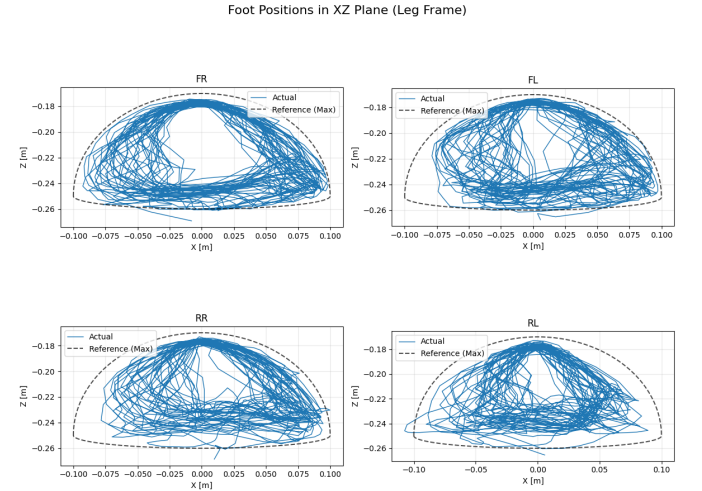


Fig. 10: Leg-frame foot XZ trajectories for a deployed task-specific policy locomoting at 0.6 m/s.

literature, models trained on flat ground should accelerate learning on obstacles by providing initial stable locomotion priors. Another key objective would be training policies for higher velocities. Finally, since the policy was trained on a model of a physical robot with domain randomization (ground friction and observation noise), a natural next step would be Sim-to-Real deployment to validate the robustness of the learned behaviors.

V. CONCLUSION

In this mini-project, we implemented a CPG-based locomotion pipeline for a simulated quadruped in PyBullet. A network of coupled Hopf oscillators generated leg-synchronised phase signals, which were mapped to desired Cartesian foot

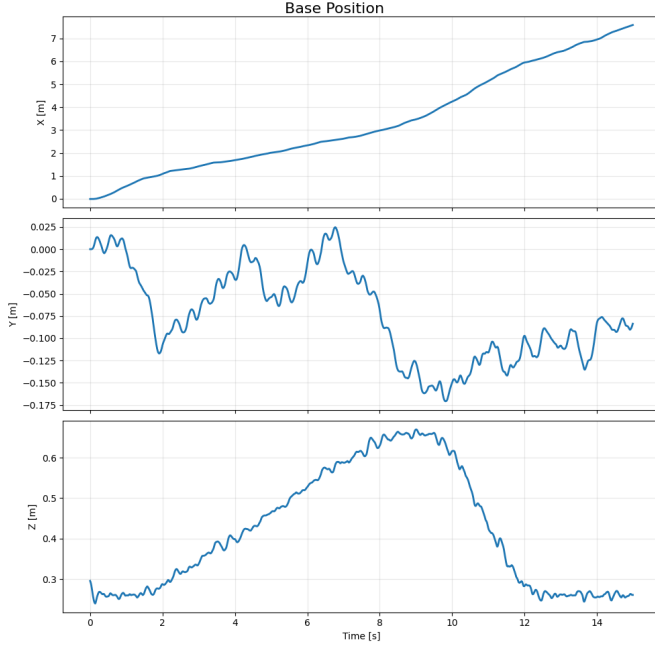


Fig. 11: Base position for a deployed task-specific policy locomoting at 0.6 m/s.

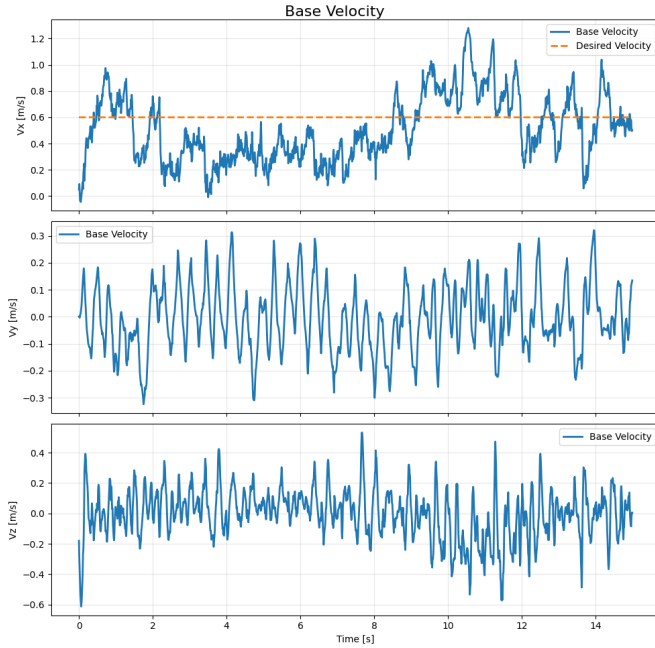
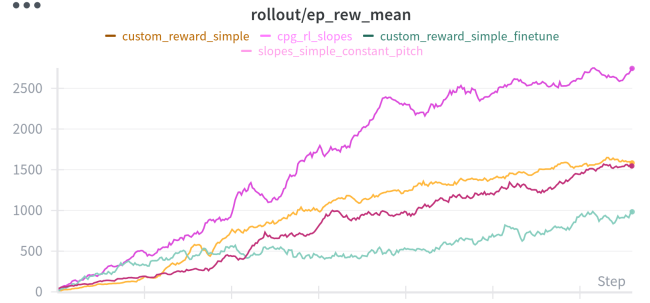
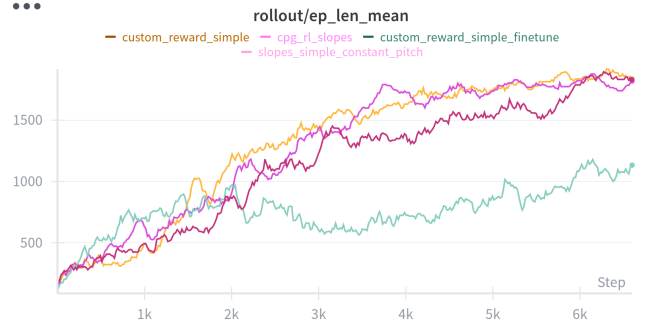


Fig. 12: Base velocity for a deployed task-specific policy locomoting at 0.6 m/s. The climbing and descending phases are clearly visible in the X-velocity plot. Throughout the episode, Y and Z velocities remain stable, indicating that terrain changes did not destabilize the robot.



(a) Mean episode length.



(b) Mean reward.

Fig. 13: Comparison of mean episode length and reward for policies trained with simple custom reward (fine-tuned vs. scratch), and scratch training with CPG-RL reward.

trajectories in the leg frame. These trajectories were tracked in torque control using joint-space PD, optionally augmented with a Cartesian PD term computed via the Jacobian transpose.

The CPG formulation provided a compact and intuitive interface to shape locomotion through a small set of interpretable hyperparameters, including swing/stance frequencies and step length. This enabled systematic speed modulation while preserving gait coordination through oscillator coupling. At the tracking level, combining Cartesian PD with joint PD introduced an explicit end-effector error feedback, which is expected to improve foot trajectory tracking and reduce steady-state deviations, especially during dynamic swing phases.

Beyond the presented baseline, the controller could be extended by adding additional feedback loops (e.g., body attitude regulation, ground reaction force feedback, or slip detection) and by incorporating higher-level descending commands to modulate gait, heading, speed, and foot placement online. Such extensions would move the current open-loop trajectory generator towards more robust and adaptive locomotion under disturbances and terrain variability.

GENERATIVE AI DECLARATION

REFERENCES

- [1] G. Bellegarda and A. Ijspeert, “Cpg-rl: Learning central pattern generators for quadruped locomotion,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.00458>

- [2] N. Heess, D. TB, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. M. A. Eslami, M. Riedmiller, and D. Silver, “Emergence of locomotion behaviours in rich environments,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.02286>
- [3] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” 2022. [Online]. Available: <https://arxiv.org/abs/2109.11978>

APPENDIX A

OBSERVATION SPACE DETAILS

TABLE II: Observation Space Specification

Observation	Dim	Lower Bound	Upper Bound
Base Orientation (Quat)	4	−1.0	1.0
Base Linear Velocity	3	−19.0	19.0
Base Angular Velocity	3	−5.0	5.0
Joint Positions (q)	N_{dof}	$q_{\min}$	$q_{\max}$
Joint Velocities ($\dot{q}$)	N_{dof}	$-v_{\text{lim}}$	v_{lim}
Foot Contacts (Boolean)	4	0.0	1.0
Last Policy Action	8	−1.0	1.0
Leg Extension (r)	4	μ_{low}	μ_{high}
Leg Ext. Rate ($\dot{r}$)	4	−5.0	5.0
Leg Angle (θ)	4	$-\pi$	π
Leg Ang. Rate ($\dot{\theta}$)	4	-9π	9π
Desired Vel. (v_x^{des})	1	$v_{x,\min}^{\text{des}}$	$v_{x,\max}^{\text{des}}$

TABLE III: Reward Function Components (Based on [1])

Component	Formula	Description
Fwd. Velocity	$0.75 \cdot \exp\left(-\frac{(v_x - v_{\text{des}})^2}{0.25}\right)$	Tracking reward for forward velocity v_{des} .
Lat. Velocity	$0.75 \cdot \exp\left(-\frac{v_y^2}{0.25}\right)$	Reward for minimizing lateral drift ($v_y \rightarrow 0$).
Yaw Rate	$0.50 \cdot \exp\left(-\frac{(\dot{\psi} - \dot{\psi}_{\text{des}})^2}{0.25}\right)$	Tracking reward for desired yaw rate $\dot{\psi}_{\text{des}}$.
Vertical Stability	$-2.0 \cdot v_z^2$	Penalty for vertical velocity to prevent bouncing.
Ang. Rate Stability	$-0.05 \cdot (\omega_x^2 + \omega_y^2)$	Penalizes roll and pitch rates.
Energy Penalty	$-0.001 \cdot \boldsymbol{\tau} \cdot \dot{\mathbf{q}} $	Penalizes mechanical power consumption.

TABLE IV: PPO Hyperparameters Configuration

Parameter	Value
Batch size	8192
Mini-batch size	2048
Number of epochs	10
Clip range	0.2
Entropy coefficient	0.0
Discount factor (γ)	0.99
GAE discount factor (λ)	0.95
Desired KL-divergence (kl_*)	0.01
Learning rate (α)	3×10^{-4} or Adaptive
Number of hidden layers	3
Number of hidden units	[512, 256, 128]
Activation function	ELU